Chapter 1

Two Dimensions

In the previous chapter, we solved a one-dimensional problem—a penny falling from the Empire State Building. Now we'll solve a two-dimensional problem—finding the trajectory of a baseball.

To do that, we'll use spatial vectors to represent quantities in two and three dimensions, including force, acceleration, velocity, and position.

1.1 Spatial Vectors

The word *vector* means different things to different people. In MATLAB, a vector is a matrix that has either one row or one column. So far, we've used MATLAB vectors to represent the following:

Sequences A mathematical sequence, like the Fibonacci numbers, is a set of values identified by integer indices; in Chapter 4.5, we used a MATLAB vector to store the elements of a sequence.

State vectors A state vector is a set of values that describes the state of a physical system. When you call `ode45`, you give it initial conditions in a state vector. Then, when `ode45` calls your rate function, it gives you a state vector.

Time series One of the results from `ode45` is a vector that represents a sequence of time values.

In this chapter, we'll see another use of MATLAB vectors: representing *spatial vectors*. A spatial vector represents a multidimensional physical quantity like position, velocity, acceleration, or force.

For example, to represent a position in two-dimensional space, we can use a vector with two elements:

```
>> P = [3 4]
```

To interpret this vector, we have to know the coordinate system it is defined in. Most commonly, we use a Cartesian system where the x-axis points east and the y-axis points north. This is one of many typical conventions, referred to in three dimensions and East-North-Up (ENU). In that case `P` represents a point 3 units east and 4 units north of the origin.

When a spatial vector is represented in this way, we can use it to compute the magnitude and direction of a physical quantity. For example, the *magnitude* of `P` is the distance from the origin to `P`, which is the hypotenuse of the triangle with sides `P(1)` and `P(2)`. We can compute it using the Pythagorean theorem:

```
>> sqrt(P(1)^2 + P(2)^2)
ans = 5
```

Or we can do the same thing using the function `norm`, which computes the *Euclidean norm* of a vector, which is its magnitude:

```
>> norm(P)
ans = 5
```

There are two ways to get the *direction* of a vector. One convention is to compute the angle between the vector and the x-axis:

```
>> atan2(P(2), P(1))
ans = 0.9273
```

In this example, the angle is about 0.9 rad. The same function gives the angle from one point to another by taking `atan2` of the differences in coordinates—for example, `atan2(y2-y1, x2-x1)` returns the bearing from point 1 to point 2.

But for computational purposes, we often represent direction with a *unit vector*, which is a vector with length 1. To get a unit vector we can divide a vector by its length:

```
function res = hat(V)
    res = V / norm(V)
end
```

This function takes a vector, `V`, and returns a unit vector with the same direction as `V`. It's called `hat` because in mathematical notation, unit vectors are written with a "hat" symbol. For example, the unit vector with the same direction as $\mathbf{P}$ would be written $\hat{\mathbf{P}}$.

The two representations of direction are interchangeable. Given an angle θ, the corresponding unit vector is $[\cos\theta, \sin\theta]$. Given a unit vector $\hat{\mathbf{P}}$, the corresponding angle is `atan2(P(2), P(1))`. Which form is easier to work with depends on the problem: forces and additions favor the vector form, while a vehicle's heading is often more natural as a single angle.

1.2 Adding Vectors

Vectors are useful for representing quantities like force and acceleration because we can add them up without having to think explicitly about direction.

As an example, suppose we have two vectors representing forces:

```
>> A = [2, 4];
>> B = [2, -2];
```

`A` represents a force pulling northeast; `B` represents a force pulling southeast, as shown in Figure 1.1:

To compute the sum of these forces, all we have to do is add the vectors:

```
>> A + B
ans = 4     2
```

Later in the chapter, we'll use vector addition to add accelerations due to different forces acting on a baseball.

1.3 ODEs in Two Dimensions

So far we've used `ode45` to solve a system of first-order equations and a single second-order equation. Now we'll take one more step, solving a system of second-order equations.

The recipe is the same regardless of where the physics comes from: assemble a state vector, write a rate function that returns $\dot{\mathbf{x}}$, and hand it to `ode45`. In this chapter the rate function comes from Newton's laws—forces produce accelerations. Other 2D models are *kinematic*: the inputs are velocities, and there is no $F = ma$ step. Many vehicle models in robotics and naval architecture take this form. Either way, the structure of the program is identical.

As an example, we'll simulate the flight of a baseball. If there is no wind and no spin on the ball, the ball travels in a vertical plane, so we can think of the system as two-dimensional, with x representing the horizontal distance traveled from the starting place and y representing height or altitude.

Using Newtonian mechanics ($F = ma$), our simple model of the dynamics of a baseball are the following equations of motion:

$$\begin{aligned} \ddot{x}(t) &= 0 \\ \ddot{y}(t) &= -g \end{aligned}$$

There are no forces (yet) in the horizontal x direction and only gravity in the vertical y direction.

Using the same process from Section 12.1 we can introduce state variable for the velocity of the baseball in the horizontal and vertical direction, v_x and v_y. This process of introducing

new variables is confusing at first, but hopefully it makes some intuitive sense that the *state* of our baseball system is described by position and velocity. The state vector of the system is then these four states. They can be in any order. The order is important to keep in mind for book keeping in implementation, but the physics don't change. We'll choose to define the state vector as

$$\mathbf{x}(t) = \begin{Bmatrix} x \\ v_x \\ y \\ v_y \end{Bmatrix}. \tag{1.1}$$

The model, now in state-space from, becomes

$$\begin{Bmatrix} \dot{x}(t) \\ \dot{v_x}(t) \\ \dot{y}(t) \\ \dot{v_y}(t) \end{Bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \begin{Bmatrix} x \\ v_x \\ y \\ v_y \end{Bmatrix} + \begin{Bmatrix} 0 \\ 0 \\ 0 \\ -g \end{Bmatrix}. \tag{1.2}$$

Listing 1.1 shows a rate function, named `state_transition`, we can use to simulate this system with `ode45`:

Listing 1.1: A rate function we can use to model the flight of a baseball

```
function xx_dot = state_transition(t, xx)
    % Unpack state into components
    x = xx(1);
    vx = xx(2);
    y = xx(3);
    vy = xx(4);

    % Acceleration in x and y from subroutine
    aa = acceleration(t, xx);

    % Equations of motion
    x_dot = vx;
    vx_dot = aa(1);
    y_dot = vy;
    vy_dot = aa(2);

    % Pack the result into a column vector
    xx_dot = [x_dot vx_dot y_dot vy_dot]';
end

function accel = acceleration(t, xx)
    % Acceleration for time (t) and state (xx)
    % Gravity
    g = 9.81;
```

```
    % Return 1x2 vector of acceleration in x and y directions
    accel = [0, -g];
end
```

The second argument is understood to be a vector, `xx`, with four elements. To avoid a hard-to-find semantic bug, it is key that you use a consistent interpretation of the meaning of this vector — that it includes the variables in the order that we defined in Equation 1.1. Having a written mathematical model, Equation 1.2, is a good defense.

The goal of the rate function is to compute the derivative of the state vector `xx`, the left-hand side of Equation 1.2.

For now, we'll ignore air resistance, so the only force on the baseball is gravity. We represent acceleration due to gravity with a vector that has magnitude `g` and direction along the negative y-axis.

Let's assume that a ball is batted from an initial position 1 m above the home plate, with an initial velocity of 40 m/s in the horizontal and 30 m/s in the vertical direction.

Here's how we can call `ode45` with these initial conditions:

```
% Simulate the flight of a baseball with no drag.
% State vector is x, v_x, y, v_y

% Initial conditions
xx_init = [0 30 1 40]';

% Solve with ode45
tspan = [0 6];
[tout, Xout] = ode45(@state_transition, tspan, xx_init);

% Extract positions
x = Xout(:, 1);
y = Xout(:, 3);

% Plot
figure(1);
clf;
plot(tout, x)
hold on;
plot(tout, y)
xlabel('Time (s)')
ylabel('Position (m)')
legend('x position', 'y position', 'Location', 'northwest')

figure(2);
clf;
plot(x, y)
```

```
xlabel('X Position (m)')
ylabel('Y Position (m)')
```

The initial conditions are defined by `xx_init` as a column vector using the transpose operator `'`.

The output variables from `ode45` are a vector, `tout`, that contains time values and a matrix, `Xout`, with four columns. Each row of `Xout` is the state of the system at the corresponding time from `tout`. We can slice this matrix to access the position values based on our model's definition of state, Equation 1.1.

Figure 1.2 shows what these coordinates look like as a function of time and in space. The x-coordinate increases linearly because the x velocity is constant. The y-coordinate goes up and down, as we expect.

The simulation ends just before the ball lands, having traveled almost 180 m. That's substantially farther than a real baseball would travel, because we have ignored air resistance, or "drag force."

1.4 Drag Force

A simple model for the drag force on a baseball is

$$\mathbf{F}_\mathrm{d} = -\frac{1}{2}\rho v C_\mathrm{d} A \hat{\mathbf{V}}$$

where $\mathbf{F}_\mathrm{d}$ is a vector that represents the force on the baseball due to drag, ρ is the density of air, C_d is the drag coefficient, and A is the cross-sectional area.

$\mathbf{V}$ is the baseball's velocity vector, v is the magnitude of $\mathbf{V}$, and $\hat{\mathbf{V}}$ is a unit vector in the same direction as $\mathbf{V}$. The minus sign at the beginning means that the result is in the opposite direction to $\mathbf{V}$.

The function in Listing 1.4 computes the drag force on a baseball:

```
function ff_d = drag_force(vv)
    % Drag force vector as a functio of velocity vector
    % Drag coefficient [dimensionless]
    C_d = 0.3;
    % Density, air [kg/m^3]
    rho = 1.3;
    % Area, cross-sectional [m^2]
    A = 0.0042;
    % Magnitude of velocity vector [m/s]
    v = norm(vv);
    % Drag force [N]
    ff_d = - 1/2 * C_d * rho * A * v * vv;
end
```

The drag coefficient for a baseball is about 0.3. The density of air at sea level is about $1.3\,\mathrm{kg/m^3}$. The cross-sectional area of a baseball is $0.0042\,\mathrm{m^2}$.

Now we have to update `acceleration` to take drag into account:

```
function accel = acceleration(t, xx)
    % Acceleration for time (t) and state (xx)

    % Gravity
    g = 9.81;

    % Drag subroutine
    ff_d = drag_force([xx(2), xx(4)]);
    % Mass [kg]
    m = 0.145;
    % Drag acceleration
    aa_d = ff_d/m;

    % Return 1x2 vector of acceleration in x and y directions
    accel = [0; -g] + aa_d;
end
```

As in Listing 1.1, `acceleration` represents acceleration due to gravity with a vector that has magnitude `g` and direction along the negative y-axis. But now it also computes drag force and divides by the mass of the baseball to get acceleration due to drag. Finally, it adds the two acceleration vectors, representing gravity and drag, to get the total acceleration of the baseball.

Figure 1.3 shows the following quantities graphically: (1) acceleration due to drag, $\mathbf{D}$, which is in the opposite direction to (2) velocity, $\mathbf{V}$; (3) acceleration due to gravity, $\mathbf{G}$, which is straight down; and (4) total acceleration, $\mathbf{A}$, which is the sum of $\mathbf{D}$ and $\mathbf{G}$.

Figure 1.4 shows the results from `ode45`. The ball lands after about 5 s, having traveled less than 120 m, substantially less than what we got without air resistance, about 180 m.

This result suggests that ignoring air resistance is not a good choice for modeling a baseball.

1.5 Chapter Review

In this chapter, we simulated the flight of a baseball with and without air resistance and saw that the difference is substantial. We can conclude that it's important to model air resistance if we want to make accurate predictions about baseballs and similar projectiles.

Here are some terms from this chapter you might want to remember.

A *spatial vector* is a value that represents a multidimensional physical quantity like position, velocity, acceleration, or force. A spatial vector has a direction and a magnitude. The

magnitude is also called the *norm* of the vector. A *unit vector* is a vector with norm 1, which is often used to represent a direction.

In the next chapter, we'll continue with the baseball example, using `fzero`, which we saw in Chapter 8, and a new tool for optimization, called `fminsearch`. We'll also see a simple way to animate the solution of a differential equation.

1.6 Exercises

Before you go on, you might want to work on the following exercises.

Exercise 1.1. When the Boston Red Sox won the World Series in 2007, they played the Colorado Rockies at their home field in Denver, Colorado. Find an estimate of the density of air in the Mile High City. What effect does this have on drag? What effect does it have on the distance the baseball travels?

Exercise 1.2. The actual drag on a baseball is more complicated than what is captured by our simple model. In particular, the drag coefficient depends on velocity. You can get some details from Robert K. Adair's *The Physics of Baseball* (Harper Perennial, 2002); the figure you need is reproduced at `https://greenteapress.com/matlab/drag`.

Use this data to specify a more realistic model of drag and modify your program to implement it. How big is the effect on the distance the baseball travels?

Exercise 1.3. According to Wikipedia, the record distance for a human cannonball is 59.05 m (see `https://greenteapress.com/matlab/cannon`).

Modify the example from this chapter to simulate the flight of a human cannonball. You might have to do some research to find the drag coefficient and cross-sectional area for a flying human.

Find the initial velocity (both magnitude and direction) you would need to break this record. You might have to experiment to find the optimal launch angle.

How much acceleration can a human withstand without injury? At this maximum acceleration, how long would the barrel of the cannon have to be to reach the initial velocity needed to break the record?

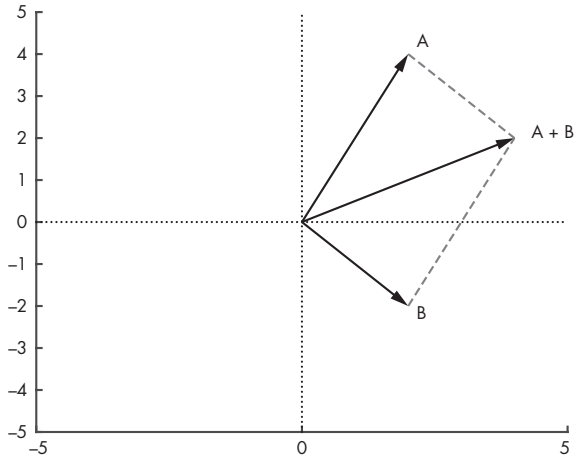
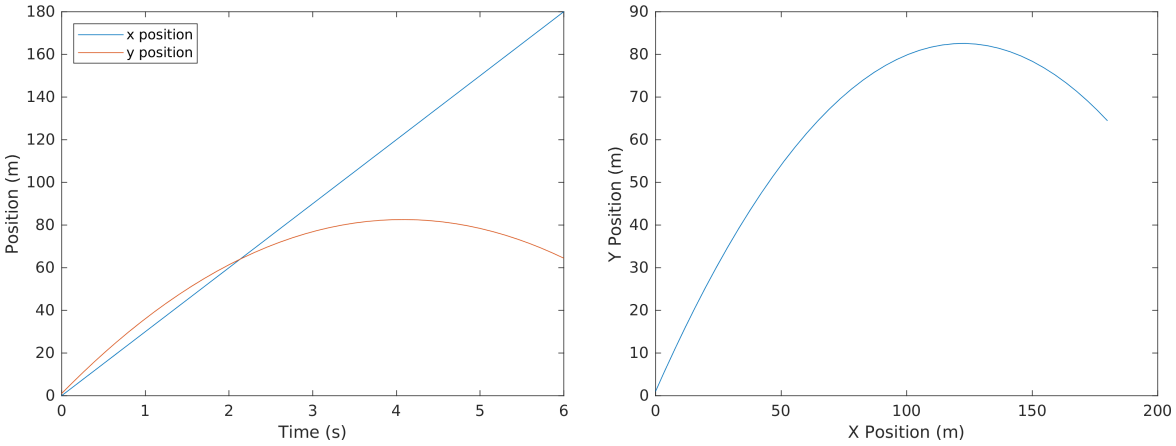


Figure 1.1: The sum of two forces represented by vectors



(a) Baseball solution time series

(b) Baseball solution in x and y

Figure 1.2: Two ways to visualize the solution

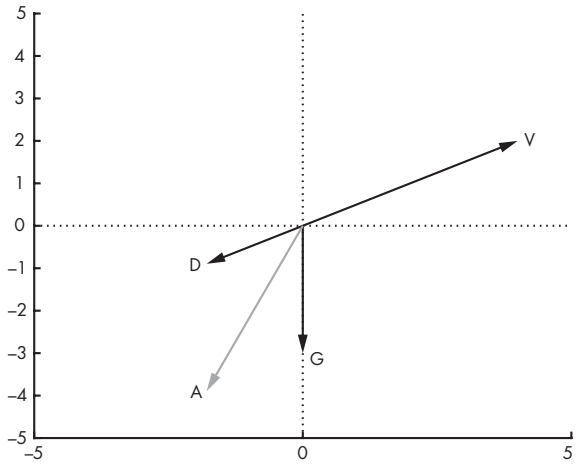
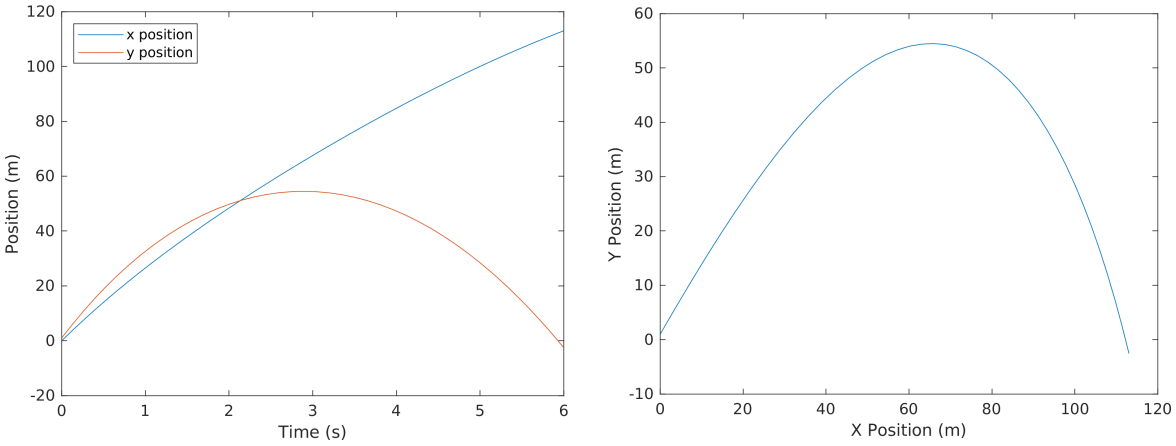


Figure 1.3: Diagram of velocity, **V**; acceleration due to drag force, **D**; acceleration due to gravity, **G**; and total acceleration, **A**



(a) Baseball solution time series

(b) Baseball solution in x and y

Figure 1.4: Simulated flight of a baseball including drag force including drag froce